



**Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)**

---

ФАКУЛЬТЕТ \_\_\_\_\_ «Информатика и системы управления»

КАФЕДРА \_\_\_\_\_ «Теоретическая информатика и компьютерные технологии»

**Лабораторная работа № 5**  
**по курсу «Разработка параллельных и распределенных**  
**программ»**  
**«Синхронизация потоков»**

Студент группы ИУ9-51Б Григорьева С. А.

Преподаватель Царев А. С.

*Москва 2025*

# 1 Задача

## Задание 1

Использование барьерной синхронизации - задача "эволюция". Дан двумерный массив клеток, каждая из которых либо содержит организм (1), либо пуста (0), изначально он заполняется случайными значениями. Каждая клетка проверяет состояние своих соседей (их 8) и изменяет своё по правилам:

- Живая клетка, вокруг которой  $< 2$  живых клеток, умирает от одиночества.
- Живая клетка, вокруг которой есть 2 или 3 живых клеток, выживает.
- Живая клетка, вокруг которой  $> 3$  живых клеток, умирает от перенаселения.
- Пустая клетка, рядом с которой равно 3 живых соседа, оживает.

Реализовать заданное количество шагов моделирования при помощи  $n$  потоков. Каждый поток должен вычислить значения в заданной ему полосе матрицы. На каждом шаге результат моделирования необходимо записывать в новую матрицу. По окончании очередного шага необходимо скопировать содержимое новой матрицы в исходную. Шаги между потоками синхронизировать с помощью барьера (ни один из потоков не должен начинать следующий шаг, пока все не закончили текущий). Учесть следующие моменты:

- у клеток, находящихся на первой строке, первом столбце, последней строке и последнем столбце, соседями являются клетки с противоположной стороны матрицы

- каждый поток видит только свою часть матрицы, поэтому если ему необходим элемент, которого в его части матрицы нет, он должен каким-либо образом составлять запрос на то, чтобы тот поток, в котором этот элемент есть, его ему предоставил.

## Задание 2

Использование синхронизаций чтения и записи. Написать программу, в которой каждый из  $n$  потоков генерирует заданное количество псевдослучайных чисел от 0 до 1000, каждое из которых он должен помещать в конец однонаправленного связного списка, но только если такого числа там ещё нет. Синхронизацию доступа к списку необходимо выполнять при помощи двух отдельных блокировок: чтения и записи. Проверку числа на принадлежность списку необходимо выполнить в блокировке на чтение. Далее, если число не обнаружено,

выполнить блокировку на запись и перед добавлением числа вновь проверить его на принадлежность списку.

## 2 Практическая реализация

Исходный код программы представлен в листингах 1– 2– 3– 4– 5– 6– 7.

Листинг 1 — Задание 1 - часть 1

```
1 package main
2 import (
3     "fmt"
4     "math/rand"
5     "sync"
6     "time"
7 )
8 const (
9     ROWS      = 1000
10    COLS       = 1000
11    STEPS      = 100
12    NUM_THREADS = 4
13 )
14 type Grid struct {
15     data [][]int
16     rows int
17     cols int
18 }
19 func NewGrid(rows, cols int) *Grid {
20     data := make([][]int, rows)
21     for i := range data {
22         data[i] = make([]int, cols)
23     }
24     return &Grid{data: data, rows: rows, cols: cols}
25 }
26 func (g *Grid) Copy() *Grid {
27     newGrid := NewGrid(g.rows, g.cols)
28     for i := 0; i < g.rows; i++ {
29         copy(newGrid.data[i], g.data[i])
30     }
31     return newGrid
32 }
33 func (g *Grid) CountNeighbors(row, col int) int {
34     count := 0
35     for dr := -1; dr <= 1; dr++ {
36         for dc := -1; dc <= 1; dc++ {
37             if dr == 0 && dc == 0 {
38                 continue
39             }
40             r := (row + dr + g.rows) % g.rows
41             c := (col + dc + g.cols) % g.cols
42             count += g.data[r][c]
43         }
44     }
45     return count
46 }
```

## Листинг 2 — Задание 1 - часть 2

```

1 func EvolutionSequential(grid *Grid, steps int) time.Duration {
2     start := time.Now()
3     current := grid.Copy()
4     next := NewGrid(grid.rows, grid.cols)
5
6     for step := 0; step < steps; step++ {
7         for i := 0; i < current.rows; i++ {
8             for j := 0; j < current.cols; j++ {
9                 neighbors := current.CountNeighbors(i, j)
10                if current.data[i][j] == 1 {
11                    if neighbors < 2 || neighbors > 3 {
12                        next.data[i][j] = 0
13                    } else {
14                        next.data[i][j] = 1
15                    }
16                } else {
17                    if neighbors == 3 {
18                        next.data[i][j] = 1
19                    } else {
20                        next.data[i][j] = 0
21                    }
22                }
23            }
24        }
25        current, next = next, current
26    }
27
28    return time.Since(start)
29 }
30 type Barrier struct {
31     count    int
32     waiting  int
33     mutex    sync.Mutex
34     cond     *sync.Cond
35 }
36 func NewBarrier(count int) *Barrier {
37     b := &Barrier{count: count}
38     b.cond = sync.NewCond(&b.mutex)
39     return b
40 }
41 func (b *Barrier) Wait() {
42     b.mutex.Lock()
43     b.waiting++
44     if b.waiting == b.count {
45         b.waiting = 0
46         b.cond.Broadcast()
47     } else {
48         b.cond.Wait()
49     }
50     b.mutex.Unlock()
51 }
52 type ThreadData struct {
53     localRows [][]int
54     startRow  int
55     endRow    int
56     threadID  int
57     mu        sync.RWMutex
58 }
59 type BorderRequest struct {
60     requestType int
61     result      chan []int
62 }

```

### Листинг 3 — Задание 1 - часть 3

```

1 const (
2   REQUEST_TOP      = 0
3   REQUEST_BOTTOM = 1
4 )
5 func EvolutionParallel(grid *Grid, steps int, numThreads int) time.
   Duration {
6   start := time.Now()
7   rowsPerThread := grid.rows / numThreads
8
9   var wg sync.WaitGroup
10  barrier := NewBarrier(numThreads)
11
12  threadData := make([]*ThreadData, numThreads)
13  borderRequests := make([]chan BorderRequest, numThreads)
14
15  for t := 0; t < numThreads; t++ {
16    borderRequests[t] = make(chan BorderRequest, 10)
17    startRow := t * rowsPerThread
18    endRow := startRow + rowsPerThread
19    if t == numThreads-1 {
20      endRow = grid.rows
21    }
22
23    localRows := make([][]int, endRow-startRow)
24    for i := 0; i < endRow-startRow; i++ {
25      localRows[i] = make([]int, grid.cols)
26      copy(localRows[i], grid.data[startRow+i])
27    }
28
29    threadData[t] = &ThreadData{
30      localRows: localRows,
31      startRow: startRow,
32      endRow: endRow,
33      threadID: t,
34    }
35  }
36
37  for t := 0; t < numThreads; t++ {
38    wg.Add(1)
39    go func(threadID int) {
40      defer wg.Done()
41
42      td := threadData[threadID]
43      localRows := td.localRows
44      localRowsNext := make([][]int, len(localRows))
45      for i := range localRowsNext {
46        localRowsNext[i] = make([]int, grid.cols)
47      }
48
49      topNeighbor := make([]int, grid.cols)
50      bottomNeighbor := make([]int, grid.cols)
51      done := make(chan bool)

```

#### Листинг 4 — Задание 1 - часть 4

```

1  go func() {
2      for {
3          select {
4              case req := <-borderRequests[threadID]:
5                  td.mu.RLock()
6                  result := make([]int, grid.cols)
7                  if req.requestType == REQUEST_TOP {
8                      copy(result, localRows[0])
9                  } else {
10                     copy(result, localRows[len(localRows)-1])
11                 }
12                 td.mu.RUnlock()
13                 req.result <- result
14             case <-done:
15                 return
16             }
17         }
18     }()
19
20     for step := 0; step < steps; step++ {
21         prevThread := threadID - 1
22         if prevThread < 0 {
23             prevThread = numThreads - 1
24         }
25         nextThread := threadID + 1
26         if nextThread >= numThreads {
27             nextThread = 0
28         }
29
30         reqTop := BorderRequest{requestType: REQUEST_BOTTOM, result:
make(chan []int, 1)}
31         reqBottom := BorderRequest{requestType: REQUEST_TOP, result:
make(chan []int, 1)}
32         borderRequests[prevThread] <- reqTop
33         borderRequests[nextThread] <- reqBottom
34         topNeighbor = <-reqTop.result
35         bottomNeighbor = <-reqBottom.result

```

## Листинг 5 — Задание 1 - часть 5

```

1      for i := 0; i < len(localRows); i++ {
2          for j := 0; j < grid.cols; j++ {
3              count := 0
4              for dr := -1; dr <= 1; dr++ {
5                  for dc := -1; dc <= 1; dc++ {
6                      if dr == 0 && dc == 0 { continue }
7                      r := i + dr
8                      c := (j + dc + grid.cols) % grid.cols
9                      var value int
10                     if r >= 0 && r < len(localRows) {
11                         value = localRows[r][c]
12                     } else if r == -1 {
13                         if topNeighbor != nil {
14                             value = topNeighbor[c]
15                         }
16                     } else if r == len(localRows) {
17                         if bottomNeighbor != nil {
18                             value = bottomNeighbor[c]
19                         }
20                     }
21                     count += value
22                 }
23             }
24             if localRows[i][j] == 1 {
25                 if count < 2 || count > 3 {
26                     localRowsNext[i][j] = 0
27                 } else { localRowsNext[i][j] = 1 }
28             } else {
29                 if count == 3 {
30                     localRowsNext[i][j] = 1
31                 } else { localRowsNext[i][j] = 0 }
32             }
33         }
34     }
35     for i := range localRows { copy(localRows[i], localRowsNext[i])
36 }
37     }
38     close(done)
39 }(t)
40 }
41 wg.Wait()
42 return time.Since(start)
43 }
44 func main() {
45     rand.Seed(time.Now().UnixNano())
46     grid := NewGrid(ROWS, COLS)
47     for i := 0; i < ROWS; i++ {
48         for j := 0; j < COLS; j++ {
49             grid.data[i][j] = rand.Intn(2)
50         }
51     }
52     seqTime := EvolutionSequential(grid.Copy(), STEPS)
53     seqTimePerStep := seqTime.Seconds() * 1000 / float64(STEPS)
54     parTime := EvolutionParallel(grid.Copy(), STEPS, NUM_THREADS)
55     parTimePerStep := parTime.Seconds() * 1000 / float64(STEPS)
56     fmt.Printf("Average execution time of one step without threads:
57                %.4f ms\n", seqTimePerStep)
57     fmt.Printf("Average execution time of one step with threads: %.4
58                f ms\n", parTimePerStep)
58 }

```

## Листинг 6 — Задание 2 - часть 1

```

1 #include <iostream>
2 #include <thread>
3 #include <mutex>
4 #include <random>
5 #include <unordered_set>
6 #include <vector>
7
8 constexpr int NUM_THREADS = 4;
9 constexpr int NUMBERS_PER_THREAD = 1000;
10 constexpr int MAX_VALUE = 1000;
11
12 struct Node {
13     int value;
14     Node* next;
15
16     Node(int val) : value(val), next(nullptr) {}
17 };
18
19 class SafeList {
20 private:
21     Node* head;
22     std::mutex read_mutex;
23     std::mutex write_mutex;
24
25 public:
26     SafeList() : head(nullptr) {}
27
28     ~SafeList() {
29         Node* current = head;
30         while (current != nullptr) {
31             Node* next = current->next;
32             delete current;
33             current = next;
34         }
35     }
36
37     bool Contains(int value) {
38         Node* current = head;
39         while (current != nullptr) {
40             if (current->value == value) {
41                 return true;
42             }
43             current = current->next;
44         }
45         return false;
46     }
47
48     bool ContainsWithMutex(int value) {
49         read_mutex.lock();
50         bool result = Contains(value);
51         read_mutex.unlock();
52         return result;
53     }

```

## Листинг 7 — Задание 2 - часть 2

```

1  bool Add(int value) {
2      write_mutex.lock();
3      if (Contains(value)) {
4          write_mutex.unlock();
5          return false;
6      }
7      Node* newNode = new Node(value);
8      if (head == nullptr) {
9          head = newNode;
10     } else {
11         Node* current = head;
12         while (current->next != nullptr) {
13             current = current->next;
14         }
15         current->next = newNode;
16     }
17     write_mutex.unlock();
18     return true;
19 }
20
21 bool CheckNoDuplicates() {
22     std::unordered_set<int> seen;
23     Node* current = head;
24     while (current != nullptr) {
25         if (seen.find(current->value) != seen.end()) {
26             return false;
27         }
28         seen.insert(current->value);
29         current = current->next;
30     }
31     return true;
32 }
33 };
34
35 int main() {
36     std::random_device rd;
37     std::mt19937 gen(rd());
38     std::uniform_int_distribution<> dis(0, MAX_VALUE);
39     SafeList list;
40     std::vector<std::thread> threads;
41     for (int t = 0; t < NUM_THREADS; t++) {
42         threads.emplace_back([&list, &gen, &dis, t]() {
43             int added = 0;
44             for (int i = 0; i < NUMBERS_PER_THREAD; i++) {
45                 int value = dis(gen);
46
47                 if (!list.ContainsWithMutex(value)) {
48                     if (list.Add(value)) {
49                         added++;
50                     }
51                 }
52             }
53         });
54     }
55     for (auto& thread : threads) {
56         thread.join();
57     }
58     return 0;
59 }

```

### **3 Характеристики устройства**

Модель оборудования: MEGABOOK T14TA (TECNO T14TA)

Процессор: 11th Gen Intel(R) Core(TM) i5-115

Память: 16 ГиБ

Емкость диска: 512 ГБ

## **4    Время работы программы**

### **1 вариант**

Среднее время выполнения одного шага без потоков: 37.9159 мс

Среднее время выполнения одного шага с потоками: 7.6362 мс